



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

6

Classe abstraite et Interface

Sofiane HACHICHA
ISIMG 2025 - 2026

6- Classe abstraite et Interface

6.1 Les classes abstraites (1/3)

POO : Java



- Une classe abstraite est introduite par le mot-clé **abstract**. Elle sert à regrouper des attributs et des méthodes qui seront utilisés par des classes dérivées concrètes.
- Une méthode abstraite, également déclarée avec le mot-clé **abstract**, ne possède pas de corps. Lorsqu'une classe contient au moins une méthode abstraite, elle est considérée comme abstraite. Cependant, il est important de noter que l'inverse n'est pas toujours vrai.
- Une classe abstraite ne peut pas être instanciée (**new**).
 - Cependant, il est possible de créer des tableaux ou des collections contenant des références à des objets de type de la classe abstraite.
- Si une classe dérivée d'une classe abstraite ne redéfinit pas toutes les méthodes abstraites, elle reste elle-même abstraite.

6- Classe abstraite et Interface

6.1 Les classes abstraites (2/3)



- Pour utiliser une classe abstraite, il est nécessaire de définir une classe **concrète** qui hérite de cette classe abstraite et qui implémente **toutes** les méthodes abstraites.

- Exemple :

```
abstract class Shape {  
    public abstract double perimeter();  
}  
  
class Cercle extends Shape {  
    private double r;  
    public Cercle(double r) { this.r=r; }  
    @Override  
    public double perimeter() { return 2 * Math.PI * r; }  
}
```





6- Classe abstraite et Interface

6.1 Les classes abstraites (3/3)

P.O.O : Java



```
class Rectangle extends Shape {  
    private double height , width;  
    public Rectangle(double height , double width ) {  
        this.height=height; this.width=width; }  
    @Override  
    public double perimeter() { return 2 * (height + width); }  
}  
  
public class TestShape {  
    public static void main(String[] args) {  
        double sum_of_perimeters = 0;  
        Shape[] shapes = { new Cercle(2), new Rectangle(2,3), new Cercle(5) };  
        for (int i = 0; i < shapes.length; i++)  
            sum_of_perimeters += shapes[i].perimeter();  
        System.out.println(sum_of_perimeters);  
    }  
}
```

Résultat d'affichage



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (1/18)

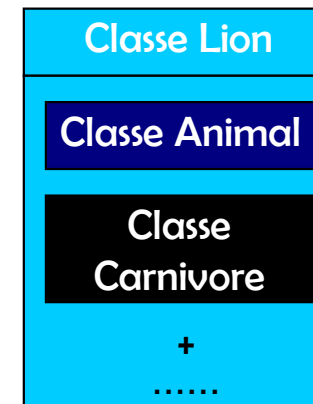
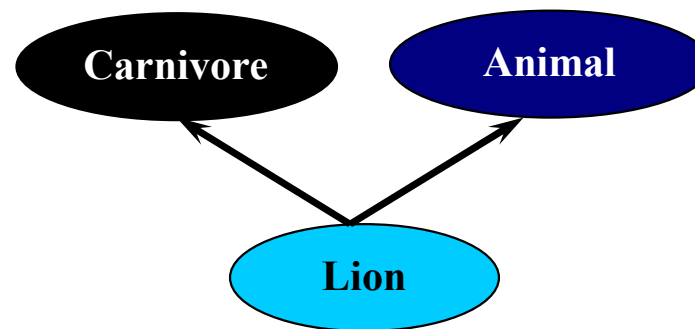
POO : Java



● Principe de l'héritage multiple :

● **Supposons** que la classe **Lion** hérite des classes **Carnivore** et **Animal**.

- Une instance de **Lion** contient donc les attributs et les méthodes de la classe **Carnivore**, les attributs et les méthodes de la classe **Animal**, et enfin les attributs et les méthodes spécifiques concernant un **Lion**.



6- Classe abstraite et Interface

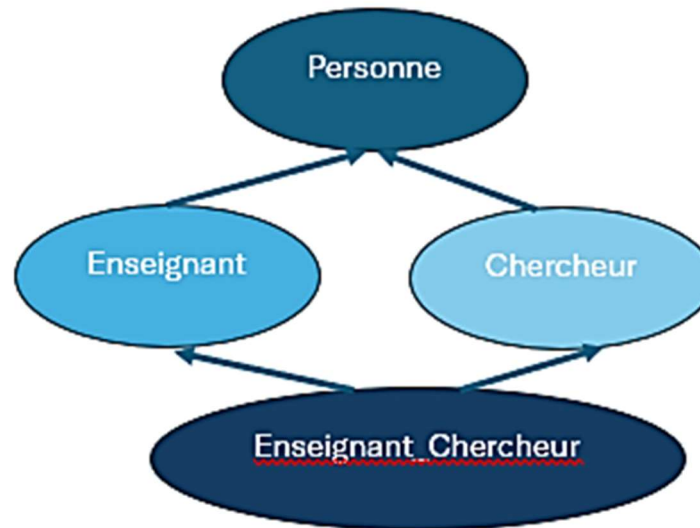
6.2 L'héritage et les interfaces (2/18)

P.O.O : Java



● Problème de l'héritage multiple en diamant :

- Un attribut de la classe **Personne** apparaît en **double** dans la classe **Enseignant_Chercheur**.
- Si une méthode de la classe **Personne** est redéfinie dans la classe **Enseignant** et dans la classe **Chercheur**, quelle est celle qui est utilisée dans la classe **Enseignant_Chercheur** ?



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (3/18)



● Principe de l'interface :

- Une interface est introduite par le mot-clé **interface**. Avant **Java 8**, une interface se comportait comme une classe dont les méthodes étaient **publiques** et **abstraites** et dont tous les attributs étaient **static** et **final**.
 - Les mots-clés **static**, **final**, **abstract** et **public** étaient implicites et donc facultatifs dans la définition d'une interface.
- Une interface est un moyen de préciser les services qu'une classe peut rendre. C'est un modèle d'**implémentation**. Elle est inutilisable en elle-même.
- Une interface ne peut pas être instanciée. Elle est plutôt **implémentée par une classe concrète**.
- Une classe concrète, qui implémente une interface, **doit** définir les corps des méthodes abstraites de l'interface.

6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (4/18)



- Une interface est utilisée lorsque nous avons besoin de garantir qu'une classe possède certaines méthodes et se comporte d'une manière définie. Par exemple :
 - On peut imaginer une interface **AppareilElectrique** qui définit les méthodes **estEnclenche()** et **alimente(boolean)**.
 - Si une classe concrète implémente l'interface **AppareilElectrique**, on sait donc qu'elle possède au moins ces deux méthodes, qu'il s'agisse d'une radio, d'une lampe ou d'un autre appareil.
 - Lorsque l'on souhaite indiquer à un appareil qu'il est alimenté, on exécute **app.alimente(true)** ; et pour lui indiquer qu'il a été débranché, on exécute **app.alimente(false)**.



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (5/18)

P.O.O : Java



● Exemple :

● Interface **AppareilElectrique**

```
public interface AppareilElectrique
{
    double freqMiseEnRoute = 96.5; // 96.5 MHz freq;

    /* teste si l'appareil est enclenché */
    public boolean estEnclenche( );

    /* Cette méthode est invoquée lorsque l'appareil est branché à
    une source de courant active, avec true, ou false si la source de
    courant est inactivée */
    public void alimente(boolean alim);
}
```



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (6/18)

P.O.O : Java



- Classe **Radio** qui implémente cette dernière interface

```
class Radio implements AppareilElectrique
{
    double freq;
    boolean allumee = false;

    public boolean estEndlenche() { return allumee; }
    public void alimente(boolean a)
    {
        allumee = a;
        if (allumee) { freq = freqMiseEnRoute; }
    }

    public boolean changeFreq(double freq)
    {
        if (allumee == true) { this.freq = freq; return allumee; }
        return false;
    }
}
```



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (7/18)

P.O.O : Java



- Le mot clé « **implements** » indique que la classe Radio doit forcément fournir une définition précise des méthodes abstraites déclarées dans l'interface AppareilElectrique.

- Une classe concrète peut implémenter plusieurs interfaces.

- Elle doit ainsi fournir des définitions précises de toutes les méthodes abstraites déclarées dans les interfaces.

```
interface Truc
{
    void e();
}
```

```
interface Bidule
{
    void f();
}
```

```
class Machin implements Truc, Bidule
{
    public void e( ) {- - - }
    public void f( ) {- - - }
}
```

- Plusieurs classes différentes peuvent implémenter de manières différentes une même interface.

- Plusieurs réalisations possibles d'une même spécification (**polymorphisme**).



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (8/18)

- Une classe concrète peut hériter d'une seule classe et implémenter une ou plusieurs interfaces.
- Un record peut implémenter une ou plusieurs interfaces.
- Une interface peut hériter de plusieurs autres interfaces.
- Le problème de **l'héritage multiple en diamant** peut être résolu à l'aide des **interfaces**.

```
interface Personne{- - -}  
interface Chercheur extends Personne {- - -}  
class Enseignant implements Personne {- - -}  
class Enseignant_Chercheur extends Enseignant implements Chercheur {- - - }
```



6- Classe abstraite et Interface

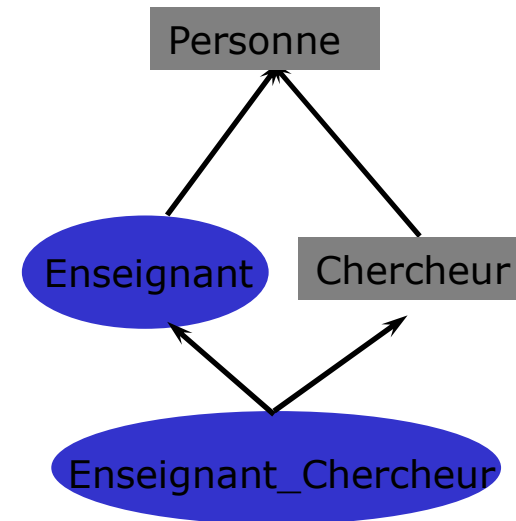
6.2 L'héritage et les interfaces (9/18)

POO : Java



● Solution au problème de l'héritage multiple en diamant :

- Les attributs de l'interface **Personne** ne sont pas redéfinis dans **Enseignant** ni **Chercheur**
 - Ils sont déclarés **final**.
- Les méthodes de l'interface **Personne** sont implémentées dans **Enseignant**, voir redéfinies dans **Enseignant_Chercheur**, mais elles ne peuvent pas être implémentées dans **Chercheur** qui est une interface.



 interface

 classe



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (10/18)

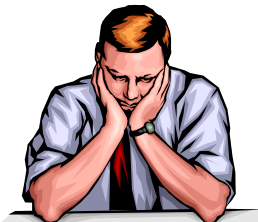
P.O.O : Java



- Une classe concrète qui implémente une interface doit donc disposer d'une implémentation pour toutes les méthodes abstraites de cette interface mais également pour toutes les méthodes abstraites des interfaces dont cette dernière hérite.
- Dans l'exemple ci-contre, on considère une classe concrète Z qui implémente deux interfaces X et Y :

```
interface X {- - - -}
interface Y {- - - -}
class Z implements X, Y {- - - -}
```

- Les attributs de **X** et **Y** sont **final** donc pas de risque de duplication des attributs de **X** et de **Y** dans **Z**.
- Et pour les méthodes ?



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (11/18)



● Conflit de nom avec les interfaces :

- Si les méthodes **f** déclarées dans les interfaces **X** et **Y** ont des paramètres de type différents et n'importe quel type de retour.

- La classe **Z** aura **deux implémentations** de la méthode **f** qui correspondent aux deux versions distinguées par les paramètres.

```
interface X
{
    void f(int i);
}
```

```
interface Y
{
    void f(char c);
}
```

```
class Z implements X,Y
{
    public void f(int i) {...}
    public void f(char c) {...}
}
```



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (12/18)

POO : Java



● Conflit de nom avec les interfaces :

- Si les méthodes **f** déclarées dans les interfaces **X** et **Y** ont les mêmes types de paramètres et le même type de retour.

- La classe **Z** aura **une seule implémentation** de **f** correspondant aux deux spécifications identiques.

```
interface X
{
    void f();
}
```

```
interface Y
{
    void f();
}
```

```
class Z implements X,Y
{
    public void f() {...}
}
```



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (13/18)

POO : Java



● Conflit de nom avec les interfaces :

- Si les méthodes **f** déclarées dans les interfaces **X** et **Y** ont les mêmes types de paramètres et des types de retour différents.

- la classe **Z** ne peut pas implémenter ces deux interfaces.

```
interface X
{
    void f(int i);
}
```

```
interface Y
{
    int f(int i);
}
```

```
class Z implements X,Y
{
    ...
}
```





6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (14/18)

● A partir de **Java 8**, une interface peut également comporter des **méthodes statiques** et des **méthodes par défaut**.

- Une interface peut fournir une implémentation par défaut de ses méthodes. Ainsi, si une classe concrète qui implémente cette interface n'implémente pas une méthode par défaut, c'est le code de l'interface qui s'exécutera.
 - Dans une interface, une méthode par défaut doit obligatoirement avoir le mot-clé **default** dans sa signature.
- Une méthode statique appartient **uniquement** à l'interface dans laquelle elle est définie : il est possible d'invoquer cette méthode statique que sur l'interface, pas sur la classe implémentant cette interface.
 - Une interface et une classe concrète qui l'implémente peuvent avoir deux méthodes statiques ayant la même signature et l'une ne remplace pas l'autre.





6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (15/18)

P.O.O : Java



```
interface MyInterface {  
    default void defaultMethod(){  
        System.out.println("Méthode1");  
    }  
    static void staticMethod(){  
        System.out.println("Méthode statique");  
    }  
}  
  
public class MyClass implements MyInterface {  
    public void defaultMethod(){  
        System.out.println("Méthode2");  
    }  
    public static void main(String[] args) {  
        new MyClass().defaultMethod();  
        MyInterface.staticMethod();  
        MyClass.staticMethod();  
    }  
    static void staticMethod(){  
        System.out.println("Une autre méthode statique"); }  
}
```

Résultat d'affichage





6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (16/18)

```
interface X {  
    default int f()  
    { return 1;}  
}  
interface Y {  
    default int f()  
    { return 2;}  
}  
public class Z implements X, Y {  
    public int f(){ // Obligation de redéfinir la méthode par défaut f()  
        return Y.super.f();  
    }  
    public static void main(String[] args) {  
        X obj = new Z();  
        System.out.println(obj instanceof Y);  
        System.out.println(obj.f());  
    }  
}
```

Résultat d'affichage



6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (17/18)

P.O.O : Java



● Une **interface fonctionnelle** est une interface qui ne contient qu'une seule méthode abstraite.

- Une interface fonctionnelle peut contenir des méthodes par défaut et statiques qui ont une implémentation, en plus de la méthode unique abstraite.
- Java fournit une annotation **@FunctionalInterface**, qui est utilisée pour déclarer une interface comme interface fonctionnelle.

```
@FunctionalInterface
interface TestI {
    public void show();
    public default int addition(int a, int b) {
        return a + b;
    }
}
```

● Une **interface de marqueur** est une interface vide qui n'a ni champs ni méthodes. Elle indique au compilateur que les objets de la classe qui implémentent cette interface sont différents et qu'ils doivent être traités différemment.

6- Classe abstraite et Interface

6.2 L'héritage et les interfaces (18/18)



● A partir de **Java 9**, il est possible d'ajouter des **méthodes privées** et des **méthodes statiques privées** dans une interface.

- Elles améliorent la réutilisation du code à l'intérieur des interfaces et offrent le choix d'exposer uniquement les implémentations de méthodes prévues aux utilisateurs.
- Elles ne sont accessibles que dans leur interface uniquement et ne peuvent pas être accédées ou héritées d'une interface vers une autre interface ou classe. Une méthode d'interface privée ne peut pas être abstraite.

```
interface PrivateI {
    default void add(int a, int b){
        sub(a,b);
    }
    private void sub(int a, int b){
        System.out.println(a-b);
    }
}
```